

Architettura tecnica

Questo documento accompagna la nota e il README: descrive come è costruito il prototipo, componente per componente e nodo per nodo, per chi vuole leggerlo o riprodurlo. Il perché delle scelte di prodotto è nella nota (docs/report/nota.md); qui c'è il come.

1. Vista d'insieme

Tre parti, tre responsabilità:

```
browser (React, Vercel)
  | HTTPS, solo webhook
  ▼
n8n self-hosted (VPS Hostinger, Docker)
  | rete interna Docker
  ▼
cv-service (Python, FastAPI, OpenCV, Docker)
  | filesystem del repository clonato
  ▼
dataset/  experiments/  frontend/public/snapshot/

└─ orchestrazione: quattro workflow,
   │ tutte le chiamate ai modelli,
   └─ le chiavi API
└─ tutto ciò che è deterministico:
   │ correzioni, controllo di fedeltà,
   └─ dataset, record, giudizi, tabellone
```

Il sito non conosce nessuna chiave e non parla mai con i modelli: chiama i webhook di n8n. n8n chiama Anthropic (Claude Haiku 4.5) e Replicate (SDXL + ControlNet, Real-ESRGAN) con le credenziali salvate nella propria istanza, e chiama cv-service sulla rete interna (http://cv-service:8000 , mai esposto su internet). cv-service legge e scrive sul repository clonato sul server: i record delle esecuzioni in experiments/runs/ , le risposte dello Studio in experiments/judgments.csv , le immagini prodotte in dataset/processed/ .

Un solo vincolo di progetto attraversa tutto: le correzioni ammesse sono quelle per cui esiste un parametro in EnhanceParams (cv-service/app/schemas.py , docs/contracts.md). Un modello non può chiedere di sostituire un cielo o togliere un oggetto perché non esiste il parametro; il solo flusso che tocca i pixel liberamente, il generativo, passa dallo stesso controllo di fedeltà degli altri.

2. I tre flussi

Quello che si confronta è chi decide cosa correggere. Le correzioni le esegue sempre lo stesso sotto-workflow (Correggi), così la differenza misurata è solo nella diagnosi.

- **D1, regole.** `POST /prepare` misura la foto (luminanza media, deviazione standard, dominante di colore, rumore stimato, inclinazione delle linee verticali, avvisi su compressione e risoluzione) e `auto_params` traduce le misure in parametri con regole fisse. Nessuna chiamata a modelli.
- **D2, modello con verifica.** Claude Haiku riceve la foto e le stesse misure, restituisce un JSON con difetti (vocabolario chiuso), `exposure` in stop, `white_balance`, `clahe_clip`, `denoise`, `rotate_deg`, `sharpen`, `recommendation`. Il nodo Piano converte e applica le guardie deterministiche (sotto). Dopo Correggi, il modello rivede originale e risultato affiancati: se risponde `adjust` con un nuovo piano, Correggi gira una seconda volta.
- **D3, generativo.** SDXL + ControlNet (canny) su Replicate in modalità `img2img`, `strength 0.35`, prompt "the same room, real estate listing photo, well exposed, natural white balance, no noise, sharp and clean, realistic, unchanged furniture, walls and layout", seed fisso. Nessuna diagnosi, nessun modulo: l'immagine torna a cv-service, che la confronta con l'originale e misura a posteriori cosa è cambiato.

Le guardie deterministiche sul piano del modello (`plan_js`)

Il piano di D2 passa da quattro correzioni automatiche prima di arrivare a Correggi, ognuna registrata in `plan_fixes` e contata nel tabellone:

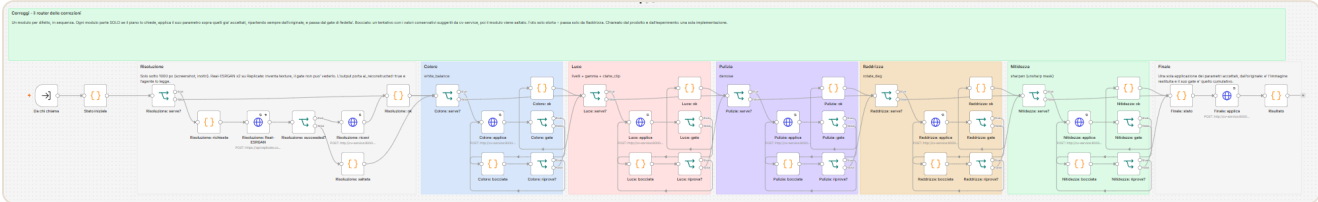
1. se il rilevatore di linee ha misurato un'inclinazione, quel valore sostituisce il `rotate_deg` del modello (un modello di visione capisce "pende a sinistra", il segno dell'angolo molto meno, e il gate non può accorgersi di un verso sbagliato perché allinea l'originale prima di confrontare);
2. se il piano scurisce una foto buia o schiarisce una foto chiara, la gamma viene invertita (errore di segno, non di giudizio);
3. su input compresso `clahe_clip` va a 0 e `sharpen` a non più di 0,2 (il contrasto locale amplifica i blocchi JPEG sui muri piatti);
4. `exposure` in stop diventa $\text{gamma} = 2^{(-\text{exposure}/2)}$: il modello ragiona in "più chiara di uno stop", non in gamma, che i modelli invertono di frequente;
5. se l'inclinazione misurata supera i 17°, nessuna rotazione (il ritaglio necessario butterebbe via troppa foto): il piano azzerà `rotate_deg`, segna il difetto `tilt` e aggiunge il consiglio `reshoot`. Vale anche per le regole (D1).

Se il modello non risponde o il JSON non è leggibile, il piano ricade su quello delle regole e la versione viene marcata `error` con `source: heuristic`: l'agente riceve comunque una foto.

3. I workflow n8n, nodo per nodo

I quattro workflow sono generati da `n8n/build_workflows.py` : i prompt vengono da `prompts/diagnosis.md` (sezioni System, User, Review), gli indirizzi da `.env` . Le sticky note sul canvas sono generate anch'esse e descrivono ogni corsia. Le convenzioni: i nodi Code girano "una volta per item"; i nodi HTTP verso cv-service hanno timeout 180 s e fino a 8 tentativi a 15 s di distanza; i nodi verso Anthropic nel prodotto hanno `onError: continue` , così un errore del modello non ferma il flusso ma diventa una ricaduta sulle regole.

3.1 Correggi (workflow-correct.json , 50 nodi, sotto-workflow)



Il canvas di Correggi: sei corsie, ognuna con serve? → applica → gate → ok / bocciata → riprova?

Riceve `{ input: { image_b64, image_id?, low_resolution }, plan, save_as }` e restituisce `{ output_b64, output_path, params, fidelity, crop_pct, steps, applied, changed, ai_reconstructed }` .

nodo	tipo	cosa fa
Da chi chiama	Execute Workflow Trigger	riceve l'item dal chiamante (Prodotto o Batch)
Stato iniziale	Code	costruisce lo stato che attraversa le corsie: <code>plan</code> completato con i valori neutri, <code>accepted</code> (parametri accettati finora, all'inizio solo <code>recommendation</code>), <code>steps</code> (sei righe, una per modulo), <code>retry</code>

Poi sei corsie. La prima, **Risoluzione**, è l'unica generativa e parte solo se `input.low_resolution` è vero (lato lungo sotto 1000 px):

nodo	tipo	cosa fa
Risoluzione: serve?	IF	<code>input.low_resolution === true</code>
Risoluzione: richiesta	Code	corpo per Replicate: Real-ESRGAN x2, <code>face_enhance: false</code> , immagine inline o URL del dataset
Risoluzione: Real-ESRGAN	HTTP	<code>POST api.replicate.com/v1/predictions</code> con <code>Prefer: wait=60</code>
Risoluzione: succeeded?	IF	<code>status === 'succeeded'</code>
Risoluzione: ricevi	HTTP	<code>POST cv-service/gate_remote</code> : scarica l'output, lo confronta con l'input, restituisce immagine e fedeltà
Risoluzione: ok	Code	l'immagine ingrandita diventa l'input dei moduli successivi; <code>ai_reconstructed = true</code> ; il piano viene adattato (<code>denoise → 0</code> , <code>clahe_clip ≤ 1</code> , <code>sharpen ≤ 0,2</code> , perché la foto ingrandita è già liscia) e l'adattamento registrato in <code>plan_adjusted</code>
Risoluzione: saltata	Code	Replicate ha fallito: si prosegue con la foto com'è, <code>steps[0]</code> registra l'errore

Le cinque corsie deterministiche (**Colore**, **Luce**, **Pulizia**, **Raddrizza**, **Nitidezza**) hanno tutte la stessa forma, sei nodi ciascuna:

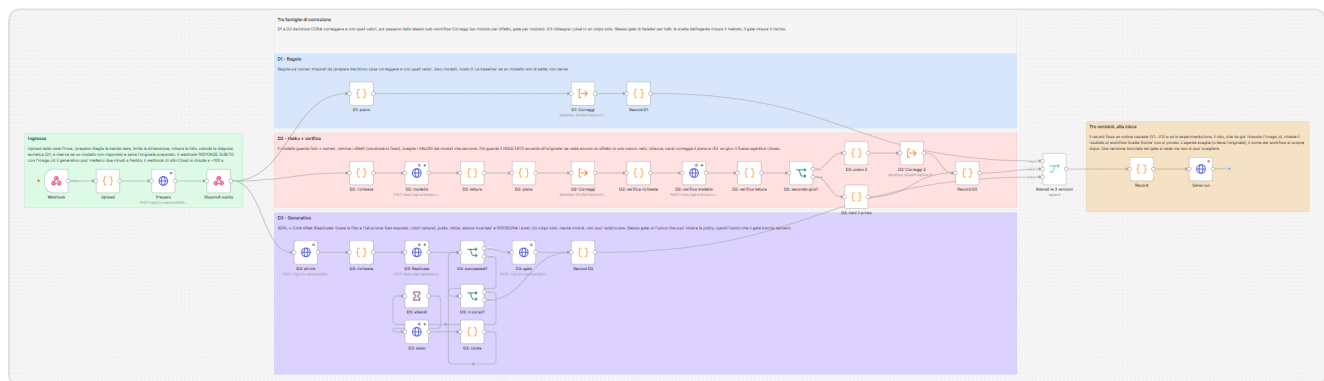
nodo	tipo	cosa fa
Modulo: serve?	IF	il piano chiede questo modulo? (<code>white_balance > 0</code> ; <code>gamma != 1 clahe_clip > 0</code> ; <code>denoise > 0</code> ; <code>rotate_deg != 0</code> ; <code>sharpen > 0</code>)
Modulo: applica	HTTP	POST <code>cv-service/apply</code> con <code>params = accepted + {parametro del modulo}</code> : si riparte sempre dall'originale e si applicano insieme i parametri già accettati più il candidato
Modulo: gate	IF	<code>fidelity.passed</code>
Modulo: ok	Code	il parametro pianificato entra in <code>accepted</code> ; la riga di <code>steps</code> registra <code>applied: true</code> , la fedeltà e i valori
Modulo: bocciata	Code	prima volta: prende i <code>suggested_conservative_params</code> che <code>cv-service</code> restituisce, li mette nel piano, segna <code>retried</code> e alza <code>_again</code> ; seconda volta: <code>applied: false</code> , <code>passed: false</code> , il modulo viene saltato e la foto resta com'era al passo precedente
Modulo: riprova?	IF	<code>_again</code> → torna ad <i>applica</i> , altrimenti passa alla corsia successiva

L'ordine delle corsie è l'ordine della pipeline di `cv-service` (bilanciamento del bianco prima della curva tonale, pulizia dopo, rotazione, nitidezza per ultima). Chiusura:

nodo	tipo	cosa fa
Finale: stato	Code	raccoglie lo stato dall'ultima corsia percorsa
Finale: applica	HTTP	POST <code>cv-service/apply</code> con tutti gli <code>accepted</code> , <code>return_image: true</code> , <code>save_as</code> per scrivere il file in <code>dataset/processed/</code>
Risultato	Code	l'oggetto restituito al chiamante: immagine, parametri, fedeltà, <code>crop_pct</code> , le sei righe di <code>steps</code> , <code>applied</code> , <code>changed</code>

Una foto solo storta passa quindi da un solo nodo *applica* (Raddrizza) più il finale; una foto senza difetti non chiama nulla e torna `changed: false` .

3.2 Prodotto (workflow-product.json , 45 nodi, webhook photo-lab-product)



Il canvas di Prodotto: ingresso, risposta immediata, tre corsie in parallelo, merge e record

nodo	tipo	cosa fa
Webhook	Webhook	POST <code>/photo-lab-product</code> , multipart con il campo <code>image</code> , CORS aperto
Upload	Code	legge il binario e lo passa in base64
Prepara	HTTP	POST <code>cv-service/prepare</code> : taglia le bande uniformi ai bordi, limita il lato lungo a 1600 px, misura, calcola parametri e difetti euristici (D1), salva l'originale preparato come <code>live_<t>_orig.jpg</code>
Rispondi subito	Respond to Webhook	<code>{ accepted: true, image_id }</code> : il browser riceve subito l'identificativo e inizia a interrogare <code>photo-lab-result</code> ogni 5 s. Necessario perché il generativo a freddo può superare i due minuti, oltre la durata di un webhook

Da qui tre corsie in parallelo.

Corsia D1 (2 nodi + Correggi): **D1: piano** (Code, `plan_js` con la diagnosi euristica di Prepara) → **D1: Correggi** (Execute Workflow, attende il sotto-workflow) → **Record D1** (Code, costruisce il `VariantResult`).

Corsia D2 (16 nodi):

nodo	tipo	cosa fa
D2: quattro versi	HTTP	<code>POST cv-service/turns</code> : la foto girata di 0, 90, 180, 270 gradi, in miniatura
D2: verso richiesta	Code	la domanda al modello: quattro immagini, "qual è quella dritta?", una lettera di risposta
D2: verso modello	HTTP	chiamata ad Anthropic (circa 1.100 token, 1,5 s)
D2: verso lettura	Code	lettera → quarto di giro (<code>orientation</code>); posta come scelta il modello non sbaglia (16/16 sul banco), come angolo sbaglia direzione una volta su due
D2: richiesta	Code	corpo per Anthropic: system e user da <code>prompts/diagnosis.md</code> , foto in base64, misure di Prepara in JSON, <code>max_tokens 600</code> , thinking disabilitato
D2: modello	HTTP	<code>POST api.anthropic.com/v1/messages</code> , credenziale <code>anthropicApi</code> di n8n
D2: lettura	Code	<code>parse_js</code> : estrae il JSON dalla risposta, calcola il costo dai token a prezzo di listino, converte <code>exposure</code> in gamma; risposta illeggibile → <code>error</code> e piano nullo
D2: piano	Code	<code>plan_js</code> : piano del modello se c'è, altrimenti riserva sulle regole; applica le guardie; l'orientamento viene dalla domanda a quattro vie, non dalla diagnosi
D2: Correggi	Execute Workflow	prima correzione
D2: verifica richiesta	Code	corpo della verifica: ORIGINAL e CORRECTED come immagini, il piano e le righe dei moduli eseguiti, prompt Review
D2: verifica modello	HTTP	seconda chiamata ad Anthropic
D2: verifica lettura	Code	stesso parser; <code>verdict ok</code> o <code>adjust</code> con un nuovo piano
D2: secondo giro?	IF	<code>verdict === 'adjust' && plan !== null</code>
D2: piano 2	Code	nuovo piano con le stesse guardie
D2: Correggi 2	Execute Workflow	seconda correzione
D2: tieni il primo	Code	verdetto <code>ok</code> : il risultato della prima correzione passa avanti
Record D2	Code	<code>VariantResult</code> con <code>iterations 1 o 2</code> , costo e latenza sommati sui due giri, <code>review_verdict</code>

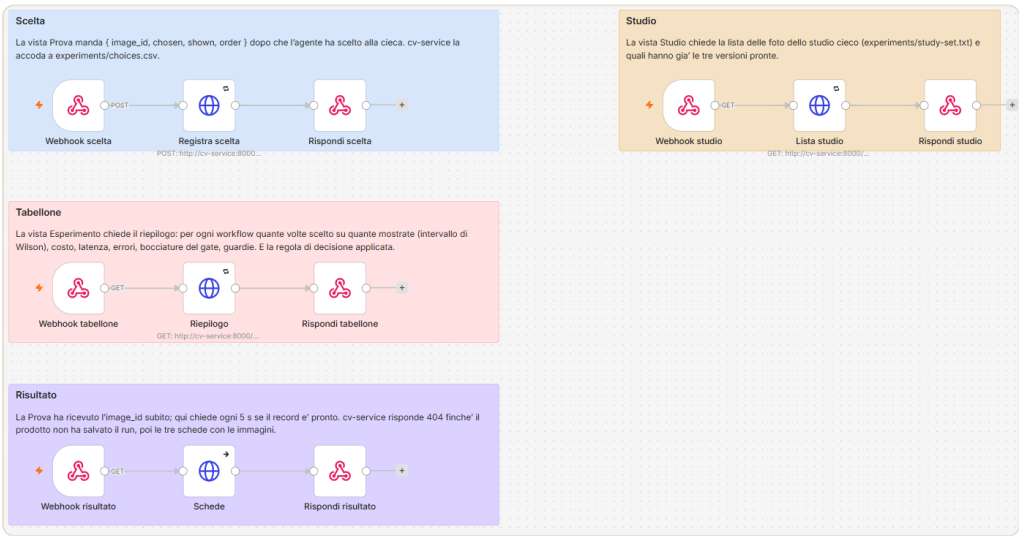
Corsia D3 (10 nodi):

nodo	tipo	cosa fa
D3: shrink	HTTP	<code>POST cv-service/prepare</code> con <code>max_side 1024</code> : Replicate accetta immagini inline fino a circa 256 KB; il modello lavora sulla copia ridotta, il gate sull'originale pieno
D3: richiesta	Code	corpo per Replicate: versione del modello, prompt e negative prompt, <code>img2img</code> , <code>strength 0.35</code> , <code>condition_scale 1.1</code> , <code>guidance_scale 5</code> , 30 passi, seed 42
D3: Replicate	HTTP	<code>POST api.replicate.com/v1/predictions</code> con <code>Prefer: wait=60</code> , credenziale header <code>Authorization: Bearer</code>
D3: succeeded?	IF	<code>status === 'succeeded'</code>
D3: in corso?	IF	<code>starting</code> o <code>processing</code> e meno di 12 tentativi
D3: attendi	Wait	15 secondi
D3: stato	HTTP	<code>GET urls.get</code> della prediction
D3: conta	Code	incrementa <code>_polls</code> e torna a <i>succeeded?</i> (il ciclo di polling del cold start, fino a 3 minuti)
D3: nsfw? / D3: nuovo seed	IF / Code	il filtro di sicurezza del modello scatta a vuoto su letti e vestiti ("NSFW content detected") e un 429 arriva con due upload insieme: fino a due nuovi tentativi con un altro seed, poi errore
D3: gate	HTTP	<code>POST cv-service/gate_remote</code> : scarica l'output, lo confronta con l'originale, lo salva come <code><id>_D3.jpg</code> , restituisce fedeltà e <code>measured_changes</code>
Record D3	Code	<code>VariantResult</code> senza diagnosi: costo dal <code>predict_time</code> a tariffa GPU, sei righe di <code>steps</code> ricavate dalle misure a posteriori (una riga è <code>applied</code> quando la variazione misurata supera quella di una semplice ricodifica), <code>status accepted</code> , <code>rejected_fidelity</code> , <code>timeout</code> o <code>error</code>

Chiusura:

nodo	tipo	cosa fa
Attendi le 3 versioni	Merge (3 ingressi)	aspetta i tre Record
Record	Code	il record dell'esecuzione: <code>image_id</code> , <code>source: live</code> , le tre <code>variants</code> senza i pixel, un <code>order</code> casuale (l'ordine cieco usato dalle schede), avvisi sull'input, difetti euristici, latenza totale
Salva run	HTTP	<code>POST cv-service/runs/<image_id></code> : scrive <code>experiments/runs/<id>.json</code>

3.3 Scelte (workflow-choice.json , 16 nodi, quattro webhook)

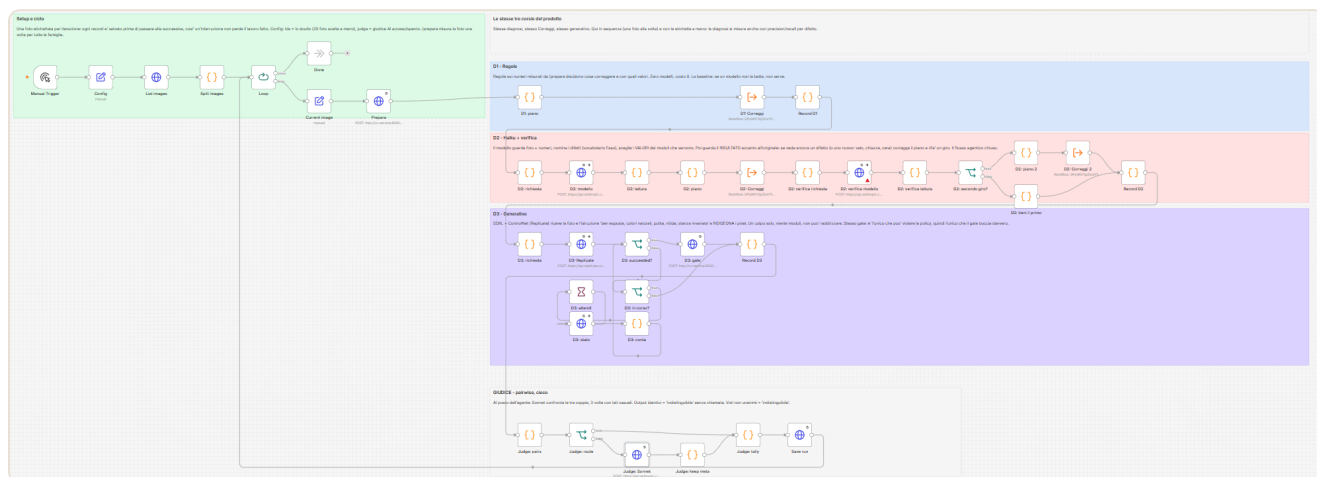


Il canvas di Scelte: quattro webhook, ciascuno inoltra a cv-service

Quattro catene identiche di tre nodi (Webhook → HTTP verso cv-service → Respond to Webhook), con CORS aperto:

webhook	metodo	cv-service	serve a
photo-lab-choice	POST	POST /choices	una risposta dello Studio (task realism, quality o best)
photo-lab-summary	GET	GET /summary	il tabellone della vista Esperimento
photo-lab-result	GET	GET /runs/{image_id}/cards	le tre versioni di un'esecuzione, per la Prova (polling) e per lo Studio
photo-lab-study	GET	GET /study	gli id dello studio, quali sono pronti, i valutatori finora

3.4 Batch (workflow-batch.json , 47 nodi, avvio manuale)



Il canvas di Batch: ciclo sulle foto, le stesse corsie in sequenza

Le stesse corsie del Prodotto, in sequenza anziché in parallelo, dentro un ciclo sulle foto del dataset:

nodo	tipo	cosa fa
Manual Trigger	Trigger	avvio a mano dal canvas
Config	Set	cv_url1 , limit , skip_done , ids (vuoto = tutto il dataset, altrimenti gli id dello studio)
List images	HTTP	GET cv-service/dataset
Split images	Code	un item per foto, filtrati da Config.ids
Loop	Split in Batches	una foto alla volta; all'uscita Done
Current image	Set	l'id corrente
Prepara	HTTP	come nel Prodotto, ma dalla foto del dataset
D1 → Record D1 → D2 ... → Record D2 → D3 ... → Record D3		le corsie del Prodotto, concatenate: ogni Record aggiunge la propria versione a quelle precedenti
Record	Code	il record della foto: le tre versioni in ordine casuale (l'ordine cieco dello Studio), source: batch
Save run	HTTP	POST cv-service/runs/<image_id> e ritorno al Loop

Nel batch le immagini raggiungono i modelli per URL del dataset (GET cv-service/dataset/<id>/file) invece che inline. Nessun giudice automatico: i giudizi sono quelli delle persone, nello Studio.

4. cv-service

FastAPI, Python 3.12, OpenCV. Trentacinque test (`pytest`). Configurazione in `app/config.py` (`max_side 1600` , `jpeg_quality 90` , `REPO_ROOT` sovrascrivibile: sul server è `/data`).

4.1 Endpoint

endpoint	usato da	cosa fa
<code>GET /health</code>	deploy	stato
<code>POST /prepare</code>	Prodotto, Batch, D3 shrink	ridimensiona, taglia le bande, misura (<code>ImageStats</code>), calcola <code>heuristic_params</code> e <code>heuristic_defects</code> , opzionalmente salva <code>shrink</code>
<code>POST /apply</code>	Correggi	applica <code>EnhanceParams</code> all'immagine, esegue il controllo di fedeltà contro l'originale allineato, restituisce immagine (se richiesta), parametri effettivi, <code>fidelity</code> , <code>crop_pct</code> e, se bocciata, <code>suggested_conservative_params</code>
<code>POST /gate_remote</code>	D3, Risoluzione	scarica un'immagine da URL, la confronta con l'originale, la salva, restituisce <code>fidelity</code> e <code>measured_changes</code>
<code>POST /runs/{id}</code>	Prodotto, Batch	salva il record; i pixel eventualmente inline vengono scritti in <code>dataset/processed/</code> e tolti dal JSON
<code>GET /runs/{id}/cards</code>	Scelte	le schede di un'esecuzione nell'ordine cieco del record, con le immagini in base64; se <code>dataset/processed/</code> manca (server appena clonato) le immagini vengono dalla copia statica del sito
<code>GET /dataset , /dataset/{id} , /dataset/{id}/file , /processed/{name}</code>	Batch, modelli	il dataset e le immagini prodotte
<code>POST /choices</code>	Scelte	accoda una riga a <code>experiments/judgments.csv</code> ; valida <code>task</code> e <code>answer</code>
<code>GET /study</code>	Scelte	id di <code>experiments/study-set.txt</code> , quali hanno un record, risposte per valutatore
<code>GET /summary</code>	Scelte	il tabellone (sotto)

4.2 La pipeline (`app/core/pipeline.py`)

`apply(img, params)` esegue, nell'ordine: bilanciamento del bianco gray-world pesato da `white_balance` ; livelli automatici (nero al percentile 0,5) e gamma; CLAHE con `clahe_clip` ; denoise non-local means con forza `denoise` scalata sulla risoluzione; rotazione di `rotate_deg` con ritaglio al rettangolo pieno (il `crop_pct` viene riportato); maschera di contrasto con `sharpen` , preceduti da `turn` (quarto di giro senza perdita, `orientation`). `analyze` produce le misure; `estimate_tilt` usa le linee verticali (Canny + Hough), con le orizzontali solo come riserva a consenso stretto, e con linee da un lato solo accetta soltanto inclinazioni fino a 3° (una prospettiva non è una rotazione); se non trova nulla chiede una seconda opinione ai segmenti più lunghi (LSD, che segue anche spigoli a basso contrasto): se, tolti due valori anomali, pendono tutti dalla stessa parte di almeno 3° e concordano entro 8°, quella è la rotazione, fino a 15° (ritaglio del 36%); oltre i 17° viene riportata ma non applicata. `auto_params` sono le regole di D1: su input compresso spegne CLAHE, alza il denoise e limita la nitidezza.

4.3 Il controllo di fedeltà (`app/core/metrics.py`)

Due misure, calibrate sul dataset (`docs/gate-calibration.md` , `experiments/scripts/calibrate_gate.py`):

- **struttura**: correlazione normalizzata per blocchi di 32 px sulla luminanza sfocata; invariante alle trasformazioni affini locali dell'intensità, che è ciò che fanno gamma, CLAHE e bilanciamento del bianco. Si usano il decimo percentile (`structure`) e il blocco peggiore (`structure_local_min`);

- **tonalità**: correlazione degli istogrammi di hue dopo normalizzazione gray-world di entrambe le immagini, per intercettare sostituzioni di colore estese.

$\text{score} = 0,7 \cdot \text{structure} + 0,3 \cdot \text{hue_corr}$; passa se $\text{score} \geq 0,90$ e $\text{structure_local_min} \geq 0,65$. Per i moduli il confronto è con l'originale ruotato della stessa quantità; per il generativo è diretto. Sul banco di calibrazione le 50 correzioni lecite passano tutte, e delle 100 vietate (oggetto rimosso, finestra sostituita, crepa riparata, smear generativo) ne passano 7, tutte crepe sottili. Per questo la super-risoluzione, che inventa texture sotto la scala del gate, è ammessa solo sotto i 1000 px e sempre etichettata `ai_reconstructed`.

4.4 Il tabellone (`app/api/experiment.py`)

`GET /summary` rilegge a ogni chiamata i record delle foto dello studio e `judgments.csv` e calcola, per flusso: tasso di alterazione (risposte «sì» al realismo, con intervallo di Wilson al 95%); voto medio di qualità e deviazione; quota di vittorie in «foto migliore» (vittorie / apparizioni, con intervallo di Wilson); tasso di pubblicabilità (per ogni coppia valutatore-versione, voto ≥ 4 e nessuna alterazione); e i parametri di supporto (costo medio per foto dai token e dal tempo GPU, tasso di errore, ricadute sulle regole, piani corretti dalle guardie, correzioni fermate dal gate). Poi applica la regola di decisione: esclusi i flussi con errori, correzioni fermate o alterazioni oltre il 10%; con almeno 30 giudizi «foto migliore» vince la quota più alta; se gli intervalli si sovrappongono decide il costo per foto. La vista Esperimento mostra le quattro misure e la regola.

5. Il sito (frontend/)

photo-lab

La foto giusta per l'annuncio

Prova

Studio

Esperimento

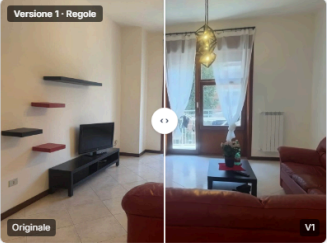
Stessa foto, tre metodi

Cosa ha fatto ogni metodo

Trascina la maniglia su ogni immagine: a sinistra l'originale, a destra la versione. Sotto, la scheda: come decide il metodo, cosa ha visto, quali moduli ha eseguito e con quali valori, tempo di elaborazione.

Un'altra foto

Versione 1 - Regole



Originale

V1

D1 - Regole

regole sui numeri misurati decidono moduli e valori; nessun modello, costo 0

Nessun modello: luminosità, colore, rumore e linee vengono misurati e regole scritte a mano scelgono le correzioni.

Cosa ha visto

colore falsato

storta

compressa

ritaglio 10%

Cosa ha fatto

● Risoluzione

non necessario

● Colore

white_balance 0.56

● Luce

non necessario

● Pulizia

denoise 4

● Raddrizza

rotate_deg 2.99 · ritaglio 10%

● Nitidezza

sharpen 0.2

Versione 2 - Modello con verifica



Originale

V2

D2 - Modello con verifica

il modello diagnostica, sceglie i valori, poi guarda il risultato e se serve corregge il piano

Un modello di visione guarda la foto e le misure, indica i difetti e i valori; poi riguarda il risultato e, se serve, corregge il piano.

Cosa ha visto

colore falsato

compressa

ritaglio 10%

Cosa ha fatto

● Risoluzione

non necessario

● Colore

white_balance 0.6

● Luce

non necessario

● Pulizia

denoise 4

● Raddrizza

rotate_deg 2.99 · ritaglio 10%

● Nitidezza

sharpen 0.2

4 moduli · fedeltà 0.997

27.3 s

guardie del sistema: 1

Versione 3 - Generativo



Originale

V3

D3 - Generativo

SDXL + ControlNet ridisegna i pixel in un colpo solo; stesso controllo di fedeltà, nessun modulo

Un modello generativo riceve la foto e un prompt da fotografo e restituisce una nuova immagine della stessa stanza.

Cosa ha visto

non diagnostica: ridisegna tutto

ricostruita da AI

Cosa ha fatto

Immagine rigenerata in un colpo solo, senza moduli. Differenze rispetto all'originale misurate dopo, sul pixel:

● Colore

dominante 10.8 → 10.4

● Luce

luminosità +0.6 · contrasto -1

● Pulizia

rumore 0.37 → 0.54

● Raddrizza

inclinazione 2.88° → 2.82°

● Nitidezza

×0.35 rispetto all'originale

● Risoluzione

scala ×0.72

La vista Prova: tre metodi affiancati con la scheda

photo-lab
La foto giusta per l'annuncio

ProvaStudioEsperimento

Foto 1 di 23 - domanda 1 di 7 - valutatore PSRealismo

Vedi elementi finti, generati o strutturalmente diversi rispetto all'originale?

Originale

Versione elaborata

No, è la stessa stanzaN

Sì, qualcosa è cambiatoS

La vista Studio, domanda sul realismo

photo-lab
La foto giusta per l'annuncio

ProvaStudioEsperimento

Esperimento - fase 1, sul prototipo

Nessuna decisione, per ora

23 foto elaborate da ogni metodo · 0 risposte da 0 valutatori · aggiornato alle 16:21 del 13/09/2026

Aggiorna

Giudizi «foto migliore»
0 / 30
raccolti / minimo richiesto per decidere

Terrebbero l'originale
•
quota di giudizi in cui nessuna versione convince

Metodo consigliato
•
in attesa dei giudizi

Valutatori
0
0 risposte in tutto

Raccolti 0 giudizi di «foto migliore» su un minimo di 30: il campione è ancora troppo piccolo per una decisione.

Regola di decisione, fissata prima di guardare i dati

- Requisito di ammissione: tasso di errore e quota di correzioni bocciate dal controllo di fedeltà sotto il 10%, e meno del 10% di risposte «alterata» nella domanda sul realismo. Un metodo che altera l'immobile non entra in gara, qualunque sia la sua qualità.
- Tra i metodi ammessi vince la foto migliore secondo i valutatori. Sotto 30 giudizi il campione non basta e non si decide; il voto di qualità (1-5) e l'output efficace descrivono il risultato ma non decidono.
- Se gli intervalli di confidenza al 95% si sovrappongono, la differenza non è significativa: a parità di qualità percepita si sceglie il metodo con costo per foto più basso. Il tempo di elaborazione non entra nella decisione: la correzione gira in background, una volta per annuncio.

Elementi qualitativi

Le quattro misure del test cieco

Ogni valutatore, nella vista Studio, risponde senza sapere quale metodo ha prodotto cosa. Realismo: originale accanto a una versione, «vedi elementi finti o diversi?». Qualità: la sola versione, voto da 1 a 5. Foto migliore: originale in alto e le versioni affiancate, «quale useresti?». Output efficace: non richiede una valutazione aggiuntiva, incrocia realismo e qualità dello stesso valutatore; efficace = voto almeno 4 su 5 e nessuna segnalazione di alterazione. Il risultato è il tasso di pubblicabilità.

Metodo	Realismo - tasso di alterazione	Qualità - voto medio (1-5)	Foto migliore - quota di vittorie	Output efficace - tasso di pubblicabilità
D1 - Regole regole sui numeri misurati decidono moduli e valori; nessun modello, costo 0	- nessuna risposta	- nessun voto	- nessun giudizio	- nessuna valutazione
D2 - Modello con verifica il modello diagnostica, sceglie i valori, poi guarda il risultato e se serve corregge il piano	- nessuna risposta	- nessun voto	- nessun giudizio	- nessuna valutazione
D3 - Generativo SDXL + ControlNet ridisegna i pixel in un colpo solo; stesso controllo di fedeltà, nessun modulo	- nessuna risposta	- nessun voto	- nessun giudizio	- nessuna valutazione

La vista Esperimento

React 19, Vite, TypeScript, Tailwind 4; nessun router, tre viste in `src/views/`. `src/lib/api.ts` conosce un solo indirizzo (`VITE_N8N_WEBHOOK_URL` , il webhook del prodotto) e ricava gli altri tre dallo stesso prefisso. Se il backend non risponde, Studio ed Esperimento leggono `public/snapshot/` (`summary.json` , `study.json` , `cards/<id>.json` , immagini ridotte), generato da `experiments/scripts/export_snapshot.py` .

- **Prova** (`ProvaView` , `VersionGrid`): `upload` → `runProduct` → polling di `photo-lab-result` ogni 5 s finché `ready` ; l'upload in corso sopravvive a un ricaricamento della pagina (`sessionStorage`). Tre colonne con `CompareSlider` (prima/dopo sulla forma dell'originale) e la scheda del metodo. `?result=<image_id>` apre un'esecuzione salvata.
- **Studio** (`StudioView`): `fetchStudy` , poi per ogni foto `fetchResult` ; le versioni con output vengono mescolate; per ciascuna realismo (tasti S/N) e qualità (1–5), poi foto migliore (1/2/3, 0 = originale). Ogni risposta è una `POST` a `photo-lab-choice` . L'avanzamento per valutatore è in `localStorage` , così si riprende dalla foto giusta. Il verdetto del gate non è mai mostrato.
- **Esperimento** (`EsperimentoView`): `fetchSummary` e le quattro misure.

6. Dati e file

percorso	cosa contiene	in git
dataset/raw/	le 24 foto (<code>img_NNN.jpg</code>) e <code>labels.csv</code> (difetti etichettati a mano)	sì
dataset/processed/	originali preparati e versioni prodotte (<code><id>_orig.jpg</code> , <code><id>_D1.jpg</code> , ...)	no
experiments/runs/<id>.json	un record per esecuzione (batch <code>img_*</code> , prodotto <code>live_*</code>)	sì
experiments/study-set.txt	gli id delle foto dello studio	sì
experiments/judgments.csv	una riga per risposta: <code>timestamp</code> , <code>tester</code> , <code>image_id</code> , <code>task</code> , <code>variant</code> , <code>answer</code> , <code>shown</code> , <code>order</code>	no (vive sul server)
frontend/public/snapshot/	copia statica per il sito	sì
n8n/workflow-*.json	i quattro workflow generati	sì
.env	chiavi e indirizzi	no (<code>.env.example</code> con i soli nomi)

7. Deploy

- **Sito:** Vercel, root `frontend`, variabile `VITE_N8N_WEBHOOK_URL`; ogni push su `main` ricostruisce.
- **n8n:** template Docker di Hostinger sul VPS, in <https>. I workflow si importano dai JSON; le credenziali (Anthropic, Replicate) si assegnano a mano ai nodi dopo l'import; Correggi va pubblicato per primo e il suo id va in `.env` come `N8N_CORRECT_WORKFLOW_ID` prima di rigenerare gli altri.
- **cv-service:** `deploy/docker-compose.yml`, container sulla rete Docker di n8n (`http://cv-service:8000`, non esposto), con `dataset/`, `experiments/` e la copia statica montati dal clone del repository in `/opt/photo-lab`. `deploy/update.sh` allinea il clone a `origin/main`, ricostruisce, riavvia e controlla `/health`. `judgments.csv` e `.env` sono ignorati da git e sopravvivono agli aggiornamenti.